MANUAL DE INSTALACIÓN Y USO

Plugin transcript-cleanup-toolkit para Claude Code

Transcripción de audio con Deepgram Nova-3 y limpieza fiel al contenido — segmentación por hablante, diagnóstico de calidad de audio y entrega de transcripciones revisadas en .docx con marcas de tiempo reales.

github.com/ramonzamora89/transcript-cleanup-toolkit

1. Qué es este plugin

transcript-cleanup-toolkit es un plugin de Claude Code que empaqueta el flujo completo para transcribir y limpiar grabaciones sueltas (entrevistas, talleres, llamadas): transcripción con Deepgram nova-3, diagnóstico de calidad de audio, reconstrucción de turnos por hablante (por *utterance* o por palabra, según cuál capture mejor la conversación), y generación de un `.docx` final de revisión con marcas de tiempo reales.

Cada grabación es un caso aparte, sin glosario de dominio ni convención de hablantes fija — todo el criterio de limpieza fina (qué es `[dudoso]`, qué se estandariza, qué nombre usar) lo aporta quien revisa cada archivo.

> **NOTA**
> El plugin no requiere que las grabaciones pertenezcan a una misma serie o proyecto temático — a diferencia de otros plugins similares con glosario fijo, este parte de cero en cada archivo.

2. Requisitos previos

- **Claude Code** instalado — CLI o Desktop App (ver sección 3).
- **Cuenta de Deepgram** con clave de API (`DEEPGRAM_API_KEY`) en el entorno donde corre Claude Code.
- `ffmpeg` — normalización de volumen y diagnóstico de eco.
- **Python 3** y **Node.js** — arman los turnos y generan el `.docx` (`npm install` dentro de `skills/transcript-cleanup/` la primera vez).

3. Instalación del plugin

El repositorio del plugin funciona como su propio *marketplace* de Claude Code: no hace falta que esté en ningún catálogo central de Anthropic, basta con que sea un repositorio público con el manifiesto `.claude-plugin/marketplace.json` — este ya lo tiene.

Desde la terminal (Claude Code CLI)

Dentro de una sesión de Claude Code, en cualquier proyecto (o sin ninguno abierto):

```
/plugin marketplace add ramonzamora89/transcript-cleanup-toolkit
/plugin install transcript-cleanup-toolkit@transcript-cleanup-toolkit
```

Verifica que quedó instalado con `/plugin`. A partir de aquí, basta con pedir en una sesión "transcribe y limpia esta entrevista" — el skill se activa solo.

Desde Claude Desktop App (Mac/Windows)

1. En el panel lateral, abrir **Directory** y seleccionar la pestaña **Plugins**.
2. Hacer clic en el botón **+** junto a los filtros ("Filter by" / "Sort by"). Se abre el diálogo **Add marketplace**.
3. Elegir **Add from a repository** ("Sync a plugin marketplace from a GitHub repository or git URL") — no "Browse Anthropic sources", que solo lista marketplaces curados por Anthropic.
4. Pegar `ramonzamora89/transcript-cleanup-toolkit` (o la URL completa) y confirmar. La app sincroniza el `marketplace.json` del repositorio.
5. El plugin aparece en la lista de Plugins — clic en el **+** de su tarjeta para instalarlo.

> **NOTA**
>
> Los comandos de barra (`/plugin ...`) y esta interfaz gráfica hacen exactamente lo mismo por debajo — la Desktop App no tiene un flujo de instalación distinto, solo una capa visual sobre el mismo marketplace.

Alternativa sin marketplace (repo clonado a mano)

```
claude --plugin-dir /ruta/local/transcript-cleanup-toolkit
```

Carga el plugin solo para esa sesión, sin pasar por GitHub.

4. Flujo de trabajo completo

Este es el recorrido de una grabación de principio a fin. Los pasos marcados como **manual** requieren criterio humano (limpieza fina); el resto son scripts del plugin.

#	Paso	Tipo
0	`scripts/transcribe.sh`	Script (llama a Deepgram)
1	`scripts/build_turns.py` y/o `build_turns_words.py`	Script (comparar ambos)
2	Limpieza fina de cada turno	Manual (criterio)
3	`cfg.json` + `scripts/build_transcript.js`	Manual + Script

Paso 0 · Transcribir

```
./scripts/transcribe.sh                    # todos los mp3 nuevos de audios/
./scripts/transcribe.sh "archivo.mp3"      # uno solo
```

Usa `model=nova-3&detect_language=true&diarize=true&utterances=true` con `smart_format` y `punctuate` activados. Para archivos `.m4a` u otro formato distinto de `.mp3`, ajustar el `Content-Type` (`audio/mp4`) — el resto del comando no cambia.

> **ANTES DE DAR POR BUENA UNA TRANSCRIPCIÓN LARGA**
>
> En archivos de más de 15-20 minutos, calcular palabras/minuto. Una conversación normal ronda 100-150 palabras/min; muy por debajo no significa que "no se habló mucho" — revisar huecos de silencio >30s. Si son frecuentes y largos, casi siempre es un problema de **volumen**, no de contenido (ver sección 6).

Paso 1 · Construir los turnos

Correr ambos scripts y comparar el número de turnos:

```
python scripts/build_turns.py <id>_deepgram.json merged.json
python scripts/build_turns_words.py <id>_deepgram.json merged_words_<id>.json
```

`build_turns.py` agrupa por `utterances` (ya segmentado por Deepgram). `build_turns_words.py` agrupa por el `speaker` de cada palabra individual. Si la versión por palabra da notablemente más turnos, es porque `utterances` está fusionando intervenciones cortas de un hablante distinto (interjecciones, preguntas breves) dentro del turno largo del otro — usarla como base en ese caso, puede incluso recuperar frases que la versión por utterance pierde del todo.

Paso 2 · Limpieza fina

Sobre cada turno reconstruido, aplicar estos principios:

- **No inventar.** Si un tramo no se entiende, no forzar una palabra. Si es plausible pero no seguro, marcar `[dudoso: hipótesis]` inline, con el **timestamp exacto** en `cfg.json` — no aproximado.
- **Confianza del ASR ≠ solo error de palabra.** Un tramo raro puede ser una palabra mal transcrita, pero también un cambio de hablante que la diarización no detectó.
- **Estandarizar con criterio, no a ciegas.** Un término mal transcrito de forma consistente se corrige sin `[dudoso]` solo si el contexto interno de la misma grabación lo confirma (se repite, o se dice completo en otra parte).
- **Confirmación humana manda.** Una corrección del usuario tras escuchar el audio es verdad, no una hipótesis más.

Se guarda como `turns_ts.json`: lista de `{i, label, ts, text}`.

Paso 3 · cfg.json y el documento final

```
node scripts/build_transcript.js transcripciones/<id>
```

Lee `cfg.json` + `turns_ts.json` del directorio dado y escribe `<id>_transcripcion.docx` ahí mismo. `cfg.json` lleva `{id, archivo, idioma, duracion, hablantes, notas, pendientes}` — `notas` es el contexto real de la grabación (sin saltos de línea, el generador no los convierte); `pendientes` es lo que de verdad quedó abierto en esa grabación específica, con timestamp por cada `[dudoso]` o estandarización hecha.

5. Diagnóstico de calidad de audio

Cuando Deepgram nova-3 devuelve poco o nada de una grabación, hay un árbol de decisión rápido de recorrer antes de ceder.

¿Es volumen bajo?

```
ffmpeg -i audio.mp3 -af volumedetect -f null -
```

Si `mean_volume` es muy negativo (visto: -29 a -35 dB en casos problemáticos), normalizar con `loudnorm=I=-16:TP=-1.5:LRA=11` y volver a transcribir. Probar primero en un fragmento de 60-90s de una zona sin habla detectada — más barato y confirma la hipótesis rápido.

¿Es eco o reverberación?

Si nova-3 devuelve confianza 0.0 en *todo* el archivo incluso tras normalizar/exportar a WAV sin comprimir, el modelo está rechazando el audio por completo — no es un problema del pipeline. Cruzar con `model=whisper-large&language=es` sobre un fragmento corto: si Whisper saca algo aunque sea parcial, confirma que hay habla real y el problema es específico de nova-3 con esa acústica.

> **WHISPER NO ES UN REEMPLAZO CONFIABLE**
> Whisper alucina texto repetitivo en tramos poco claros. Es un último recurso que necesita revisión humana pesada, no limpieza fina automática — avisar esto explícitamente antes de usarlo sobre un archivo completo.

Límite de tiempo con Whisper

El gateway de Deepgram corta requests síncronos largos con `whisper-large` (probado: timeout con un archivo de 72 min). Si hace falta usarlo, fragmentar en trozos de ~5 min y unir después.

6. Buenas prácticas — qué NO hacer

- No alinear un texto ya "limpio" contra el JSON — armar siempre los turnos directo del JSON crudo (rompe timestamps y atribución de hablante en turnos cortos).
- No tokenizar nombres de personas en este proyecto.
- No usar `redact=pii` de Deepgram — sobre-redacta y deja el contenido inservible.
- No forzar una corrección sin apoyo contextual solo para "limpiar" el `[dudoso]`.

7. Solución de problemas

"/plugin marketplace add" falla o no encuentra el repo

Repo mal escrito, o el repositorio no tiene `.claude-plugin/marketplace.json` en la raíz.

"/plugin install" dice que no encuentra el plugin

El nombre después de `@` debe ser el nombre del *marketplace* (definido en `marketplace.json`), que puede diferir del nombre del plugin.

El plugin se activa pero los scripts fallan

Falta `DEEPGRAM_API_KEY` en el entorno, o no se corrió `npm install` dentro de `skills/transcript-cleanup/`.